3B2 FPGA Programming Lab Report

Liu Zihe[1]*

[1]University of Cambridge

Abstract

This report documents the design, implementation and testing of a traffic light controller on an Altera Cyclone V FPGA. A three-state finite state machine controlling a pedestrian crossing is described in Verilog, compiled, and configured onto a DE1-SoC board via JTAG, and is improved with a vehicle lockout.

1 Traffic Light Controller Design

We design a controller for a pedestrian light-controlled crossing, satisfying the following requirements:

1. The initial state is vehicle GREEN and pedestrian RED.
2. State changes occur only on demand: a pedestrian request or a reset.
3. After a request, the sequence is: vehicle YELLOW for 5 s, then vehicle RED with pedestrian GREEN for 10 s, then back to the initial state.
4. An asynchronous reset returns to the initial state from any state.

These requirements are captured by a three-state FSM shown in Figure 1. State `G` is the idle state with vehicle GREEN and pedestrian RED. A pedestrian `request` triggers a transition to state `Y` (vehicle YELLOW), which serves as a warning to vehicles. After a 5 s timeout the FSM enters state `R` (vehicle RED, pedestrian GREEN), allowing pedestrians to cross. A 10 s timeout then returns the FSM to state `G`. An asynchronous active-low `reset` forces the FSM back to state `G` from any state.

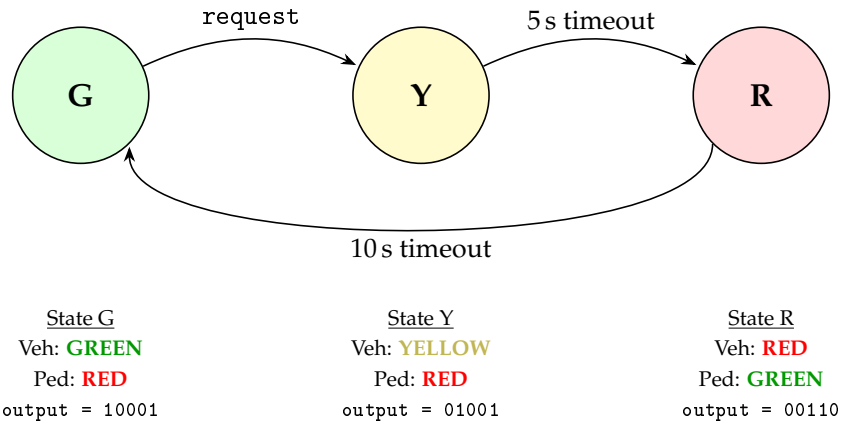

Figure 1: State diagram for the traffic light controller FSM. The 5-bit output vector is `{veh_G, veh_Y, veh_R, ped_G, ped_R}`. `reset` (active LOW, asynchronous) returns to state G from any state.

The outputs of each state and the transitions to the next states are summarised in Table 1. Each state activates exactly one vehicle light and one pedestrian light.

Table 1: State table for the traffic light controller.

	Next State				Outputs				
State	`reset`	`request`	5 s	10 s	`veh_G`	`veh_Y`	`veh_R`	`ped_G`	`ped_R`
`G`	`G`	`Y`	–	–	1	0	0	0	1
`Y`	`G`	–	`R`	–	0	1	0	0	1
`R`	`G`	–	–	`G`	0	0	1	1	0

*CRSID: zl559. Date: 20th February 2026

2 Verilog Implementation

Listing 1: Basic traffic light controller.

```
module tlc (
    input  wire        clk,
    input  wire        request,
    input  wire        reset,
    output reg  [4:0] \output   // {veh_G, veh_Y, veh_R, ped_G, ped_R}
);

  localparam G = 2'd0, Y = 2'd1, R = 2'd2;

  reg [ 1:0] state;
  reg [28:0] count;

  // Sequential: state transitions & timer
  always @(posedge clk or negedge reset) begin
    if (!reset) begin
      state <= G;
      count <= 29'd0;
    end else begin
      case (state)
        G: begin
          if (request == 1'b0) begin
            state <= Y;
            count <= 29'd0;
          end
        end
        Y: begin
          if (count == 29'd250_000_000) begin // 5s @ 50 MHz
            state <= R;
            count <= 29'd0;
          end else count <= count + 29'd1;
        end
        R: begin
          if (count == 29'd500_000_000) begin // 10s @ 50 MHz
            state <= G;
            count <= 29'd0;
          end else count <= count + 29'd1;
        end
        default: begin
          state <= G;
          count <= 29'd0;
        end
      endcase
    end
  end

  // Combinational: output mapping
  always @(*) begin
    case (state)
      G:       \output = 5'b10001;
      Y:       \output = 5'b01001;
      R:       \output = 5'b00110;
      default: \output = 5'b10001;
    endcase
  end

endmodule
```

The Verilog code in Listing 1 implements the FSM from Section 1. Key design choices made are:

- **Clock division.** The DE1-SoC provides a 50 MHz clock, but the TLC requires delays on the order of seconds. Rather than a separate clock divider module, a 29-bit counter (`count`) is used directly.
- **State encoding.** The three states are encoded with 2 bits (`G`=0, `Y`=1, `R`=2). A `default` case catches the unused encoding (3) and returns to state `G`.
- **Active-low inputs.** The DE1-SoC push-buttons output 0 when pressed. The reset is handled via `negedge reset` in the sensitivity list for asynchronous behaviour, and `request` is checked against `1'b0`.
- **Output mapping.** A combinational `always @(*)` block maps each state to a 5-bit output vector

`[4:0] = {veh_G, veh_Y, veh_R, ped_G, ped_R}`.

3 Compilation Results

Table 2: Compiler Flow Summary.

Resource	Count
Logic Elements	**TODO**
Registers	**TODO**
Pins	**TODO**

4 Pin Assignment

Table 3: Pin assignments for the DE1-SoC board.

Signal	Pin	Component
`clk`	`PIN_AF14`	50 MHz Clock
`request`	`PIN_AA15`	KEY1 (Push-button)
`reset`	`PIN_AA14`	KEY0 (Push-button)
`output[0]`	`PIN_V16`	LEDR0 (ped RED)
`output[1]`	`PIN_V17`	LEDR2 (ped GREEN)
`output[2]`	`PIN_W17`	LEDR4 (veh RED)
`output[3]`	`PIN_Y19`	LEDR6 (veh YELLOW)
`output[4]`	`PIN_W21`	LEDR8 (veh GREEN)

5 Testing

Table 4: Board testing results.

Initial State	Input Applied	Expected Next State	Pass/Fail
`G`	`request`	`Y` (5 s timer)	**TODO**
`Y`	5 s timeout	`R` (10 s timer)	**TODO**
`R`	10 s timeout	`G`	**TODO**
Any	`reset`	`G`	**TODO**

6 Improved Circuit

6.1 Updated State Machine

6.2 Verilog Implementation

6.3 Testing

7 Conclusions

A Appendix

A.1 Testbench

Listing 2: Testbench for the basic TLC.

```
// Testbench for basic Traffic Light Controller (tlc.v)
// Uses force/release on internal counter to skip long timer waits.
`timescale 1ns / 1ps

module tlc_tb;

  reg clk, request, reset;
  wire [4:0] out;

```

```
  tlc uut (.clk(clk), .request(request), .reset(reset), .\output (out));

  initial clk = 0;
  always #10 clk = ~clk; // 50 MHz

  localparam G = 2'd0, Y = 2'd1, R = 2'd2;
  localparam OUT_G = 5'b10001, OUT_Y = 5'b01001, OUT_R = 5'b00110;
  localparam Y_THRESH = 29'd250_000_000, R_THRESH = 29'd500_000_000;

  integer pass_count = 0, fail_count = 0;

  task check(input [63:0] name, input cond);
    begin
      if (cond) begin $display("  PASS: %0s", name); pass_count = pass_count + 1; end
      else      begin $display("  FAIL: %0s", name); fail_count = fail_count + 1; end
    end
  endtask

  task wait_clks(input integer n);
    integer i;
    begin for (i = 0; i < n; i = i + 1) @(posedge clk); end
  endtask

  // Skip counter to near threshold, then let it trigger naturally
  task skip_counter(input [28:0] thresh);
    begin
      force uut.count = thresh - 2;
      wait_clks(1);
      release uut.count;
      wait_clks(5);
    end
  endtask

  initial begin
    $dumpfile("tlc_tb.vcd");
    $dumpvars(0, tlc_tb);
    request = 1; reset = 1;

    // T1: Reset -> state G
    $display("\n--- T1: Reset ---");
    reset = 0; wait_clks(3);
    check("state=G", uut.state == G);
    check("out=10001", out == OUT_G);
    reset = 1; wait_clks(2);

    // T2: Full cycle G -> Y -> R -> G
    $display("\n--- T2: Full cycle ---");
    request = 0; wait_clks(2); request = 1;
    check("G->Y", uut.state == Y);
    check("out=01001", out == OUT_Y);
    skip_counter(Y_THRESH);
    check("Y->R", uut.state == R);
    check("out=00110", out == OUT_R);
    skip_counter(R_THRESH);
    check("R->G", uut.state == G);
    check("out=10001", out == OUT_G);

    // T3: Async reset from Y and R
    $display("\n--- T3: Async reset ---");
    request = 0; wait_clks(2); request = 1;
    check("in Y", uut.state == Y);
    reset = 0; wait_clks(2);
    check("Y reset->G", uut.state == G);
    reset = 1; wait_clks(2);
    // Now reset from R
    request = 0; wait_clks(2); request = 1; wait_clks(1);
    skip_counter(Y_THRESH);
    check("in R", uut.state == R);
    reset = 0; wait_clks(2);
    check("R reset->G", uut.state == G);
    reset = 1; wait_clks(2);

    // T4: Idle stability
```

```
83      $display("\n---␣T4:␣Idle␣---");
84      request = 1; wait_clks(100);
85      check("stays␣G", uut.state == G);
86
87      // Summary
88      $display("\n==␣%0d␣passed,␣%0d␣failed␣==", pass_count, fail_count);
89      if (fail_count == 0) $display("ALL␣TESTS␣PASSED");
90      else $display("SOME␣TESTS␣FAILED");
91      $finish;
92    end
93
94  endmodule
```